PROYECTO RED SOLIDARIA

Arquitectura de patrones, organización de paquetes y funcionalidades del código fuente

Informe técnico basado en la inspección del repositorio

Universidad: [Completar]

Curso: [Completar]

Autor(es): [Completar]

Docente: [Completar]

Fecha: 24 de octubre de 2026

Versión editable para revisión académica

ÍNDICE

1.1. ARQUITECTURA DE PATRONES – ORGANIZACIÓN DE LOS PAQUETES 3
1.1.1. Descripción general de la arquitectura 3
1.1.2. Arquitectura del frontend Angular 4
1.1.3. Arquitectura del backend Spring Boot 5
1.1.4. Organización de paquetes y carpetas 6
1.1.5. Patrones arquitectónicos y de diseño identificados 10
1.1.6. Comunicación entre componentes 11
1.1.7. Diagramas técnicos 12
Diagrama 1. Arquitectura general 12
Diagrama 2. Organización por módulos y capas 12
Diagrama 3. Secuencia representativa: registrar donación 12
Diagrama 4. Flujo de recepción parcial y entrega 13
1.1.8. Síntesis de la arquitectura 14
1.2. FUNCIONALIDADES DEL CÓDIGO FUENTE 15
1.2.1. Descripción funcional del sistema 15
1.2.2. Inventario de módulos y funcionalidades 15
1.2.3. Funcionalidades del frontend 17
1.2.4. Funcionalidades del backend: catálogo de endpoints 18
1.2.5. Integración con MySQL 22
1.2.6. Trazabilidad de las funcionalidades 23
1.2.7. Reglas de negocio y validaciones 24
1.2.8. Estado de implementación y limitaciones 25
1.2.9. Pruebas y verificación funcional 26

1.2.10. Síntesis funcional 27
REFERENCIAS TÉCNICAS 28

El índice se actualizará al abrir el documento en Word. Si Word pregunta si desea actualizar los campos, elija actualizar toda la tabla.

1.1. ARQUITECTURA DE PATRONES – ORGANIZACIÓN DE LOS PAQUETES

1.1.1. Descripción general de la arquitectura

Red Solidaria está organizada como una aplicación web cliente-servidor. El cliente es una SPA Angular con componentes independientes y rutas cargadas de forma diferida. El servidor es una aplicación Spring Boot que publica endpoints REST y separa funcionalidades por módulos de negocio. MySQL almacena los datos relacionales; Flyway aporta las migraciones V1 y V2, mientras Hibernate valida el esquema existente en vez de crearlo automáticamente.

Componente	Responsabilidad comprobada	Evidencia principal
Angular	Presentación, navegación, formularios, validación de interacción, guardas de rutas y clientes HTTP tipados. Las páginas standalone se cargan mediante `loadComponent`.	`frontend/src/app/app.routes.ts`, `app.config.ts` y módulos por función.
Spring Boot / Spring MVC	Autenticación, autorización, validación de entrada, aplicación de reglas de negocio, contratos DTO y respuestas HTTP.	`backend/pom.xml`, `SecurityConfiguration.java` y paquetes `modules/*`.
Persistencia	Entidades JPA y repositorios Spring Data; consultas de balance específicas mediante `JdbcClient`.	`modules/*/domain`, `modules/*/persistence` y migraciones Flyway.
Integración de desarrollo	El proxy Angular reenvía `/api/**` a `http://localhost:8080`; ambos clientes usan rutas relativas `/api`.	`frontend/proxy.conf.json` y servicios Angular.

El flujo habitual es navegador → cliente Angular → HTTP/JSON → controlador REST → servicio de aplicación → repositorio o consulta JDBC → MySQL. Las respuestas del servidor se transforman en DTO; las entidades JPA no son el contrato HTTP. No se encontró una vista HTML renderizada por Spring: la vista pertenece al frontend Angular.

> **Alcance de verificación:** el análisis se hizo sobre el código del árbol de trabajo existente. Se ejecutaron pruebas automatizadas aisladas (resultados en 1.2.9), pero no se conectó ni consultó una base MySQL real. Por ello, las afirmaciones sobre persistencia describen código y migraciones, no una certificación de despliegue o integración en vivo.

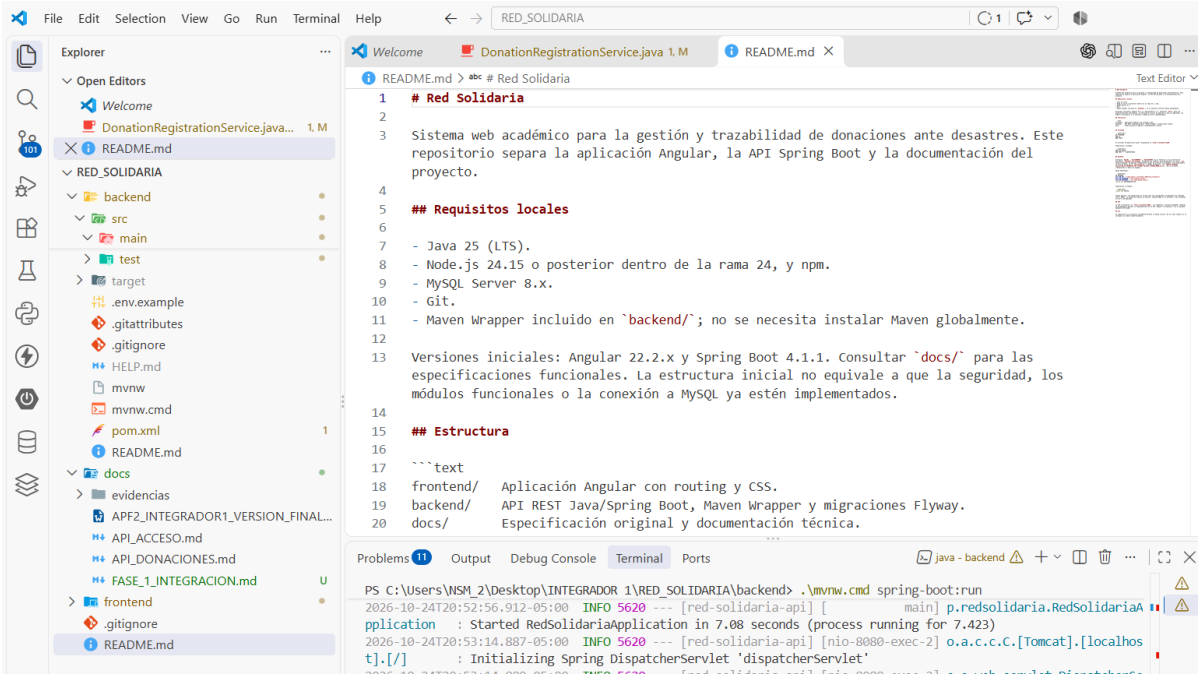


Figura 1. Evidencia E-01. Vista de VS Code con README, estructura frontend/backend/docs y versiones declaradas. Archivo relacionado: README.md. La captura es del editor real y no una imagen generada.

1.1.2. Arquitectura del frontend Angular

La configuración global de `app.config.ts` instala el router y `provideHttpClient` con el interceptor funcional `basicAuthInterceptor`. Las rutas usan componentes standalone y carga diferida. Las páginas emplean `FormsModule` y presentan estados de carga, error y éxito; las llamadas de red viven en servicios de módulo que devuelven `Observable`. No se usa almacenamiento local para mantener la sesión: `BasicAuthService` guarda credenciales y perfil en signals de memoria. El formulario de recepción conserva temporalmente una operación pendiente en `sessionStorage` para reintentos.

Carpeta/archivo	Componentes y funciones	Comunicación o dependencia	Estado observado
`app.routes.ts`, `app.config.ts`, `app.ts`	Registro de rutas, proveedores, shell de aplicación.	Router Angular, HTTP e interceptor Basic Auth.	Implementado; rutas administrativas protegidas por guards de interfaz.
`auth/`	Inicio de sesión, perfil autenticado, guards, interceptor y estado de sesión.	`GET /api/auth/validate` y `GET /api/auth/me`.	Interfaz enlazada a la API; credenciales viven en memoria y se limpian ante 401.
`donations/`	Registro, catálogos, listado/detalle administrativo, recepción y consulta pública por código.	`DonationAdminService`, `DonationTraceService`.	Flujos HTTP implementados; pruebas frontend presentes y ejecutadas.
`emergencies/`	Gestión de emergencias/zonas y componente de balance por recurso.	`EmergencyApiService`.	API enlazada; balance se representa desde la pantalla administrativa.

`needs/`	Alta de necesidades, disponibilidad de fuentes y acumulados asignados/entregados.	`NeedApiService`, servicios de emergencias y recursos.	Flujo enlazado y cubierto por pruebas de componente.
`resources/`	Consulta, alta, edición y cambio de estado de recursos; unidades de medida.	`ResourceApiService`.	Flujo HTTP implementado y cubierto en pruebas frontend.
`audit/`	Consulta paginada y filtrada de eventos.	`AuditApiService`.	Consulta enlazada; no crea eventos desde la pantalla.
`admin/`, `shared/`	Panel de navegación, shell, menú, sesión y enlaces a módulos.	Router, estado de autenticación y permisos.	Panel presente; el propio texto marca gestión de usuarios como pendiente.

Las rutas protegidas y los guards mejoran la navegación, pero no sustituyen la autorización del servidor. No existe una pantalla Angular para gestionar usuarios, roles y permisos; esa API sí está implementada en backend. El proxy descrito es de desarrollo y no configura por sí mismo el proxy de producción.

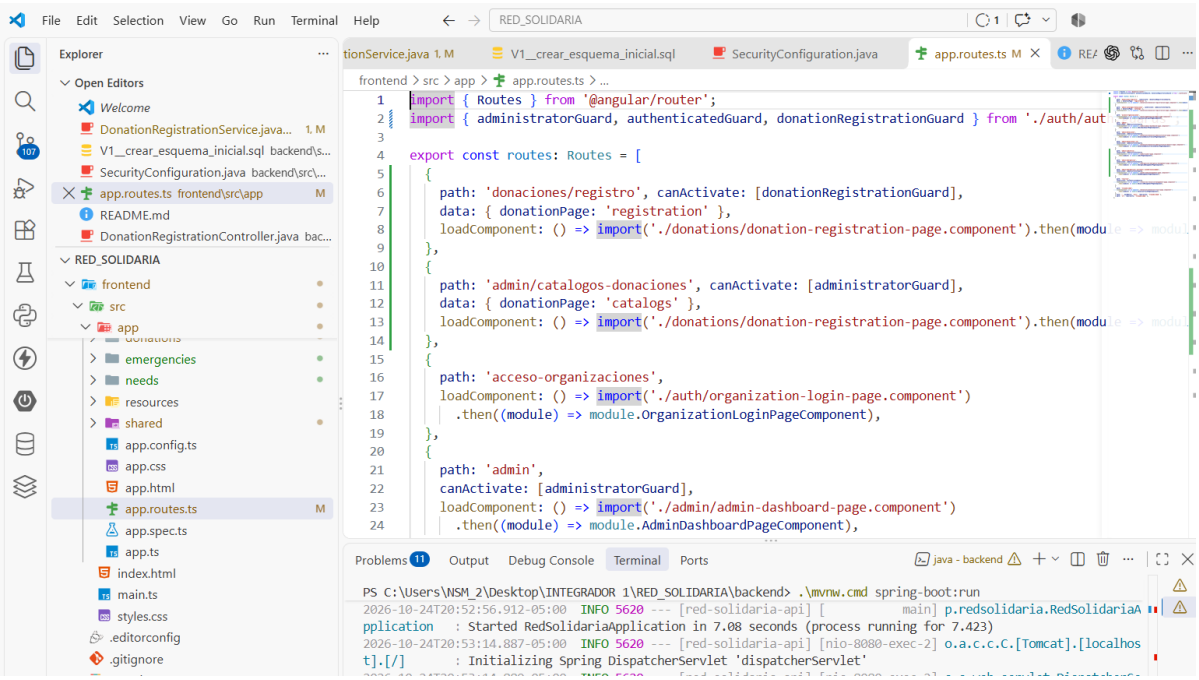


Figura 2. Evidencia E-07. Archivo `frontend/src/app/app.routes.ts`, rutas lazy y guards para registro y administración. La captura muestra código real en VS Code.

1.1.3. Arquitectura del backend Spring Boot

`RedSolidariaApplication` inicia Spring Boot. `SecurityConfiguration` configura HTTP Basic sin estado, BCrypt con coste 12, autenticación desde la cuenta/rol/permisos y reglas por autoridad. Los controladores REST validan DTO y delegan en servicios. Los servicios de aplicación coordinan entidades, repositorios y auditoría; límites transaccionales se expresan con `@Transactional`. Hibernate usa `ddl-auto=validate`, Flyway se habilita en la aplicación normal y `open-in-view=false`.

La autorización del servidor incluye `ACCESO_ADMINISTRAR` para `/api/admin/**`, permisos de registro para la creación de donaciones y acceso autenticado a recursos. La consulta pública de trazabilidad se permite solo para el GET de una ruta de un segmento. Otras rutas requieren autenticación. El perfil CLI de bootstrap es separado del servidor normal y no constituye un endpoint.

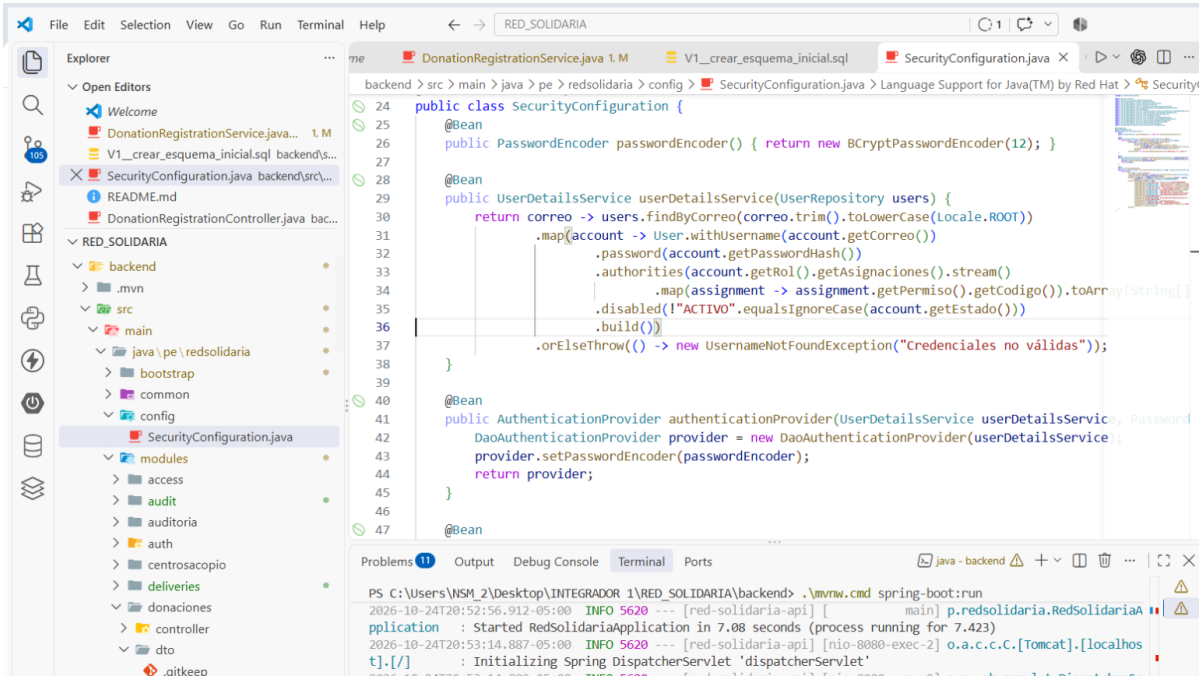


Figura 3. Evidencia E-06. Archivo `backend/src/main/java/pe/redsolidaria/config/SecurityConfiguration.java`: BCryptPasswordEncoder, servicio de usuario y provider DAO. La captura no muestra la sección de requestMatchers; sus reglas se documentan a partir del archivo inspeccionado.

1.1.4. Organización de paquetes y carpetas

La estructura efectiva es modular por área de negocio y, dentro de los módulos activos, separa API/DTO, aplicación, dominio y persistencia. Esta convención es una arquitectura por módulos con capas, no una afirmación de arquitectura hexagonal estricta: las entidades de dominio son JPA y dependen de anotaciones de persistencia.

Paquete/carpeta	Clases principales	Responsabilidad y relaciones	Evidencia
`pe.redsolidaria` `RedSolidariaApplication`	Clase de arranque.	Composición y lanzamiento del contexto Spring.	`backend/src/main/java/pe/redsolidaria/RedSolidariaApplication.java`

bootstrap	BootstrapAdminCli, BootstrapAdminService, AdminPasswordRecoveryCli, guardas y diagnósticos.	Herramientas CLI locales con comprobaciones de destino y transacciones; no son endpoints de la API normal.	backend/src/main/java/pe/redsolidaria/bootstrap/
config	SecurityConfiguration.	Beans de autenticación, codificador, filtro HTTP y políticas de autorización.	backend/src/main/java/pe/redsolidaria/config/SecurityConfiguration.java
modules/access/{api,api/dto,application,domain,persistence}	Controladores de administración/autenticación, DTO, AccessManagementService, cuentas/roles/permisos y repositorios.	Usuarios, roles, permisos y perfil autenticado. La configuración de seguridad consume UserRepository.	backend/src/main/java/pe/redsolidaria/modules/access/

modules/audit/{api,api/dto,application,domain,persistence}	AuditQueryController, AuditQueryService, AuditRecord, AuditRecordRepository.	Registro y consulta paginada/filtrada de eventos que otros módulos escriben.	modules/audit/
modules/deliveries/{api,api/dto,application,domain,persistence}	DeliveryManagementController/Service, Delivery, DeliveryLine y repositorios.	Asignaciones y entregas acumuladas relacionadas con una necesidad y una línea de donación.	modules/deliveries/
modules/donations/{api,api/dto,application,domain,persistence}	Controladores de catálogo, alta, trazabilidad, administración y recepción; DonationRegistrationService, DonationReceiptService; entidades y repositorios de donación.	Donaciones, líneas, movimientos, donantes e identidad idempotente de recepción.	modules/donations/

modules/emergencies/{api,api/dto,application,domain,persistence}	EmergencyManagementController/Service, EmergencyBalanceController, emergencia, zona, repositorios.	Gestión de emergencia/zonas y consulta de balance; el repositorio de balance usa JdbcClient.	modules/emergencies/
modules/needs/{api,api/dto,application,domain,persistence}	NeedManagementController/Service, DTO, Need, repositorio.	Necesidades por zona/recurso, saldos y fuentes donadas.	modules/needs/
modules/resources/{api,api/dto,application,domain,persistence}	Controladores de recursos/unidades, servicio, recurso, unidad y centro de acopio.	Catálogos base usados por donaciones, emergencias y necesidades.	modules/resources/

`modules/{auditoria,auth,centrosacopio,donaciones,donantes,emergencias,entregas,necesidades,recursos,reportes,trazabilidad,usuarios,zonasafectadas}`; `common`	Carpetas con marcadores `.gitkeep`; no contienen clases de implementación.	Esqueleto heredado/de reserva. No debe confundirse con las funciones reales que viven en los módulos ingleses activos. `common` también está vacío.	`backend/src/main/java/pe/redsolidaria/modules/`

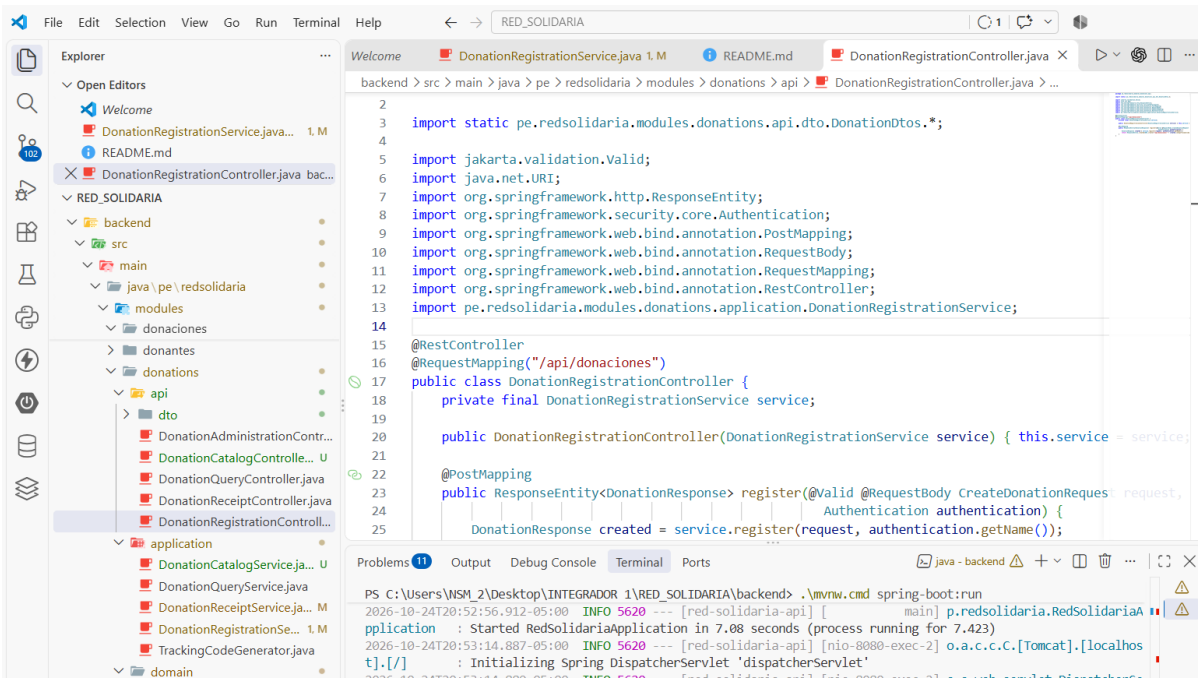


Figura 4. Evidencia E-02. Árbol real de VS Code en `modules/donations/api`, DTO y controladores; ilustra la separación por capas dentro del módulo.

1.1.5. Patrones arquitectónicos y de diseño identificados

Patrón/categoría	Evidencia comprobable	Beneficio y límite	Clasificación

Arquitectura modular por capas	Módulos de negocio con `api`, `application`, `domain` y `persistence`; por ejemplo, donaciones.	Acota responsabilidades y agrupa cambios por dominio. El acoplamiento entre módulos existe (necesidades usa repositorios de donaciones, recursos y entregas); no es modularidad aislada estricta.	Arquitectónico, confirmado.
Controller / API REST	`@RestController`, mappings HTTP y DTO; el controlador de registro delega en `DonationRegistrationService`.	Separa protocolo HTTP del caso de uso. No es MVC clásico con vistas del servidor: Angular es la vista cliente.	Organización web, confirmada con esa salvedad.
Repository	Repositorios extienden `JpaRepository`; balance usa un repositorio JDBC específico.	Centraliza persistencia y consultas. No se añadió un repositorio genérico propio.	Diseño, confirmado; Spring Data/JDBC aporta mecanismo.
Service Layer / caso de uso	Servicios `@Service` orquestan reglas, validaciones, persistencia y transacciones.	Concentra reglas fuera de controladores y delimita transacciones.	Diseño/organización, confirmado.
DTO	Records agrupados en `api/dto`, mapeados desde/hacia entidades; restricciones Jakarta Validation.	Expone contratos explícitos y evita serializar directamente entidades JPA.	Diseño, confirmado.
Inyección de dependencias / IoC	Constructores que reciben repositorios/servicios; Angular usa `inject()`. Beans gestionados por Spring.	Reduce creación manual y facilita pruebas con dobles/mocks. El alcance singleton de beans Spring es comportamiento del framework, no implementación explícita del patrón Singleton.	Mecanismo del framework, confirmado.
Transacción	`@Transactional` en altas y operaciones compuestas; bloqueos para recepción/asignación concurrente.	Coordina escrituras y rollback dentro del contexto transaccional. La concurrencia real requiere validación con MySQL y no se certifica solo por el código.	Mecanismo de persistencia, confirmado en código.
Observer/reactividad	Servicios Angular devuelven `Observable`; componentes se suscriben y usan signals para estado.	Permite composición y actualización reactiva de UI. No se encontró un Observer propio implementado por el dominio.	API del framework/RxJS; no patrón custom confirmado.
Factory, Strategy, Singleton explícito	No se identificó una implementación propia verificable de estos patrones.	No se asignan por inferirlos de nombres de clases o de beans Spring.	No confirmado.

1.1.6. Comunicación entre componentes

La interfaz invoca servicios Angular mediante rutas relativas `/api`; el proxy local apunta al servidor de Spring. El interceptor Basic Auth adjunta credenciales únicamente a peticiones de API de loopback y omite la lectura pública de trazabilidad. Si una respuesta autenticada es 401, elimina las credenciales de memoria. El backend valida autenticación y autoridad de nuevo, por lo que los guards Angular no se consideran una barrera de seguridad.

En la persistencia se mezclan JPA/Spring Data para agregados y consultas, y `JdbcClient` para el balance agregado. No se observó mensajería asíncrona, llamadas directas navegador-MySQL ni una configuración general de CORS. El proxy documentado solo es de desarrollo.

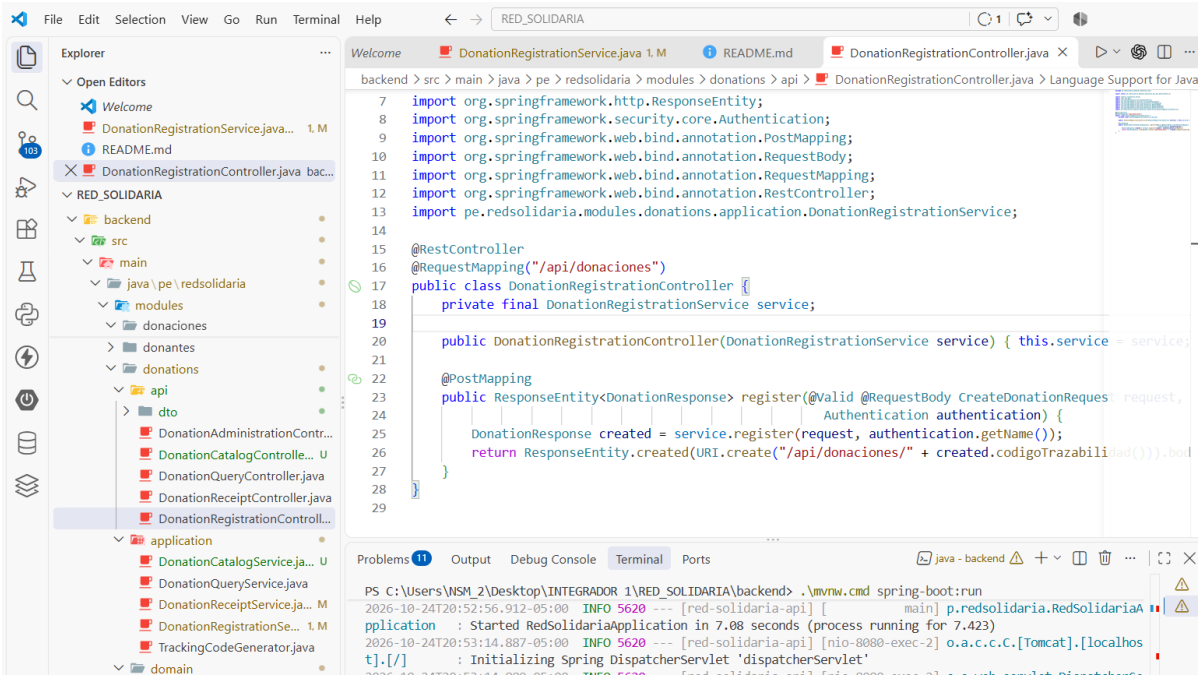


Figura 5. Evidencia E-03. `DonationRegistrationController.register`: POST JSON validado, identidad autenticada y respuesta 201 con Location de trazabilidad.

1.1.7. Diagramas técnicos

Diagrama 1. Arquitectura general

Cliente / presentación	Transporte	API / aplicación	Persistencia
Angular SPA rutas, componentes, guards servicios HttpClient	HTTP/JSON `/api/**` proxy local :4200 → :8080	Spring MVC REST SecurityConfiguration módulos y DTO servicios transaccionales	Spring Data JPA JdbcClient MySQL + Flyway

Diagrama 2. Organización por módulos y capas

```
modules/access ¬ modules/audit ⟝┤ api controllers → application services → domain/JPA
entities modules/deliveries ┼ │ │ modules/donations ⟝┤ └→ persistence repositories ⟶ MySQL
modules/emergencies┤ modules/needs ⟝⟝⟝⟝┤ (balance de emergencia: repository JdbcClient →
MySQL) modules/resources ┘ bootstrap y config son paquetes transversales; carpetas españolas
con .gitkeep no son capas activas.
```

Diagrama 3. Secuencia representativa: registrar donación

Paso	Emisor → receptor	Operación respaldada por código

1	`DonationRegistrationPageComponent` → `DonationAdminService`	La página recopila donante, emergencia o centro, ubicación, observaciones y líneas de recursos; llama al servicio del módulo.
2	`DonationAdminService.register` → API	Envía `POST /api/donaciones` como HTTP JSON; el interceptor agrega Basic Auth al API local.
3	`DonationRegistrationController.register` → servicio	Valida el DTO, usa el nombre autenticado y delega al caso de uso; responde 201 y Location.
4	`DonationRegistrationService.register` → repositorios	Valida destino, referencias, recursos activos, códigos y cantidades; persiste donación, detalles y movimiento inicial en una transacción.
5	Repositorios/JPA → MySQL	Las claves, unicidad y checks de V1 refuerzan integridad. El estado inicial es `REGISTRADA`.
6	API → Angular	Devuelve DTO de alta con código de trazabilidad; la UI presenta resultado o error HTTP.

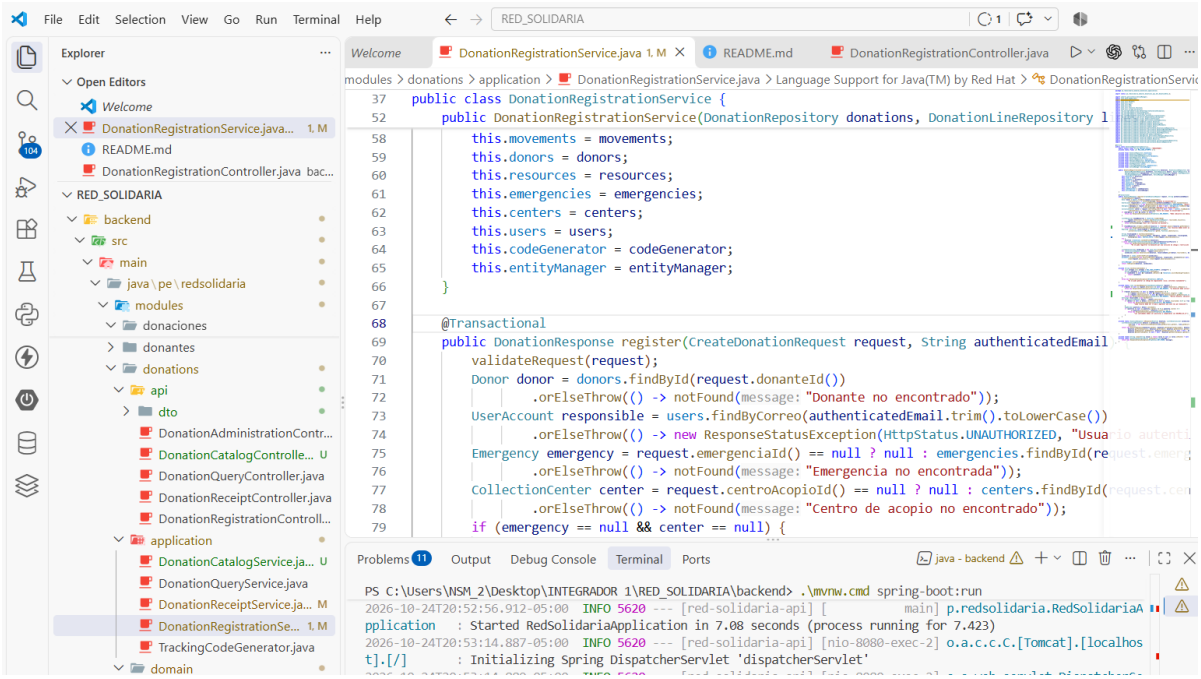


Figura 6. Evidencia E-04. `DonationRegistrationService.register`, con transacción y resolución de entidades de referencia. La captura muestra código real en VS Code.

Diagrama 4. Flujo de recepción parcial y entrega

```
POST /api/donaciones/administracion/{id}/recepciones → DonationReceiptService (READ_COMMITTED + bloqueo de donación) → cantidad recibida por línea + movimiento + auditoría + operación durable V2 → estado RECIBIDA únicamente cuando todas las líneas llegan a lo donado PUT /api/admin/necesidades/{needId}/donaciones/{sourceId}/entrega → DeliveryManagementService (bloqueo donación y necesidad) → compara totales acumulados con recibo real, recurso, emergencia y necesidad → persiste incrementos de entrega y auditoría en la transacción
```

1.1.8. Síntesis de la arquitectura

El sistema aplica una separación por cliente Angular y API REST, con módulos backend por área y capas API/aplicación/dominio/persistencia. Los patrones con evidencia son Repository, Service Layer, DTO e inyección de dependencias; transacciones y seguridad se apoyan en Spring. MVC se aplica a la frontera HTTP sin vistas de servidor. No se confirmó Factory, Strategy ni Singleton propio. La persistencia está definida por migraciones; su ejecución en un MySQL real permanece fuera de esta verificación.

1.2. FUNCIONALIDADES DEL CÓDIGO FUENTE

1.2.1. Descripción funcional del sistema

Red Solidaria implementa gestión de catálogos operativos, registro y consulta de donaciones, recepción de cantidades, emergencias y zonas, necesidades por recurso, asignación/entrega y auditoría. La aplicación incluye autenticación administrativa y una consulta pública de trazabilidad. La existencia de funcionalidades de reportes generales, notificaciones o QR no se encontró confirmada en el código revisado.

1.2.2. Inventario de módulos y funcionalidades

#	Funcionalidad	Actor / interfaz	API y lógica	Persistencia / estado
1	Inicio de sesión y perfil	Usuario institucional; `OrganizationLoginPageComponent`.	`GET /api/auth/validate`, `GET /api/auth/me`; Spring HTTP Basic y perfil con autoridades.	`usuario`, `rol`, `rol_permiso`, `permiso`. Implementado; verificación de UI en suite frontend, no prueba MySQL de autenticación en esta ejecución.
2	Administración de usuarios, roles y permisos	Solo API backend; no hay pantalla de administración identificada.	`AccessManagementController` y `AccessManagementService`; alta, estado, rol, permisos.	Mismas tablas de acceso. API implementada y servicio probado; UI parcial/pendiente. No se crea una cuenta inicial automáticamente.
3	Bootstrap y recuperación local de administrador	Operador local mediante CLI.	`BootstrapAdminCli`, `AdminPasswordRecoveryCli`, guardas de destino y servicios.	Transacción de bootstrap y BCrypt. Implementación presente; no se ejecutó CLI ni se modificó base real.
4	Recursos y unidades de medida	Administración; pantalla `ResourceManagementPageComponent`.	CRUD parcial de recursos y cambio de estado; consulta de unidades.	`recurso`, `unidad_medida`. Servicio (4 tests) y pruebas frontend ejecutados.

5	Catálogos de donantes, centros, emergencias y unidades	Formulario de donación y página de catálogos.	`DonationCatalogController/Service`, búsquedas por texto/página y creación.	`donante`, `centro_acopio`, `emergencia`, `unidad_medida`. Flujo en código; catálogo backend no incluido en el subconjunto de pruebas backend ejecutado.
6	Registro de donación	Usuario con permiso `DONACION_REGISTRAR`; formulario de registro.	`POST /api/donaciones`, genera código, estado inicial y movimiento.	`donacion`, `detalle_donacion`, `movimiento_donacion`. Pruebas de servicio (6) y frontend ejecutados; sin MySQL real.
7	Trazabilidad pública	Consulta anónima por código.	`GET /api/donaciones/{codigoTrazabilidad}`; respuesta pública omite datos del donante, IDs, observaciones privadas y responsables.	Donación, detalles y movimientos. 4 tests backend de consulta y pruebas frontend ejecutados.
8	Listado y detalle administrativo de donaciones	Administrador; listado con filtros y pantalla de detalle.	`GET .../listado` y `GET .../{id}`.	Consulta de donación, donante, destino, líneas e historial. Implementado; prueba de servicio y componentes frontend ejecutados.
9	Recepción parcial/total en acopio	Administrador; detalle administrativo.	`POST .../{id}/recepciones`; valida pendientes, acumula cantidades, crea movimiento y auditoría.	`detalle_donacion`, `movimiento_donacion`, `registro_auditoria`, V2 `recepcion_operacion`. 5 tests de servicio y 1 de dominio ejecutados; sin test MySQL de rollback aquí.
10	Emergencias y zonas afectadas	Administrador; pantalla de emergencias.	Listado, detalle, alta/edición/cambio de estado y zonas.	`emergencia`, `zona_afectada`, auditoría. 5 tests de servicio y suite frontend ejecutados.

11	Balance agregado por emergencia	Administrador; componente `EmergencyBalanceComponent`.	`GET /api/admin/emergencias/{id}/balance`, consulta agregada por recurso.	Necesidades, recibos, asignaciones y entregas. Componente frontend probado; no se ejecutó consulta contra MySQL real.
12	Necesidades y fuentes disponibles	Administrador; pantalla de necesidades dentro de zona.	Alta/listado de necesidad y consulta paginada de líneas de donación compatibles.	`necesidad`, líneas recibidas y asignaciones. 3 tests de servicio y tests frontend ejecutados.
13	Asignación a necesidad y seguimiento de entrega	Administrador; formulario de totales acumulados.	`PUT /api/admin/necesidades/{needId}/donaciones/{sourceId}/entrega`; límites, bloqueos y auditoría.	`entrega`, `detalle_entrega`, auditoría. 9 tests backend y tests frontend ejecutados; concurrencia real en MySQL no certificada.
14	Consulta de auditoría	Administrador; página de auditoría.	`GET /api/admin/auditoria` con filtros de acción/entidad/registro/usuario/fecha y paginación.	`registro_auditoria`. 3 tests de servicio y pruebas frontend ejecutados.

Los estados “implementada y verificada” en esta tabla se limitan a código enlazado y pruebas automatizadas aisladas efectivamente ejecutadas. No significan que el despliegue, MySQL real, migraciones pendientes o concurrencia de producción estén certificados.

1.2.3. Funcionalidades del frontend

Ruta	Página	Acciones / validación visible	Integración y acceso
`/acceso-organizaciones`	`OrganizationLoginPageComponent`	Correo/contraseña; valida servidor, carga perfil y traduce errores.	`/api/auth/validate`, `/api/auth/me`.
`/trazabilidad`	`DonationTracePageComponent`	Busca código y muestra estados, cantidades e historial; maneja 404 y error de conexión.	GET público; no requiere sesión.

/donaciones/registro	DonationRegistrationPageComponent	Selecciona donante, emergencia o centro, recursos/cantidades y observaciones; catálogos con filtros y paginación.	Guard donationRegistrationGuard; API de recursos, catálogos y registro.
/admin/catalogos-donaciones	Componente de registro reutilizado en modo catálogos.	Administra donantes, centros y unidades según pestaña.	Guard administrador; API catálogo.
/admin/donaciones, /admin/donaciones/:id	Lista, detalle y recepción.	Filtros código/estado; cantidades recibidas por línea; conserva la operación pendiente para reintento.	Guard administrador; GET administrativo y POST de recepción.
/admin/emergencias	Gestión de emergencias y zonas; balance embebido.	Alta/edición/estado, zonas y consulta de balance.	Guard administrador; APIs de emergencias y balance.
/admin/emergencias/:id/zonas/:zoneId/necesidades	Necesidades, fuentes y entrega.	Alta de necesidad, fuente compatible, validación de totales acumulados y errores 400/401/403/404/409.	Guard administrador; NeedApiService, ResourceApiService, EmergencyApiService.
/recursos	Gestión de recursos.	Consulta, alta, edición, estado y filtro local de texto.	Guard autenticado; escrituras protegidas de nuevo por backend.
/admin/auditoria	Consulta de eventos.	Filtros y paginación; muestra errores y eventos.	Guard administrador; AuditApiService.

La pantalla inicial redirige a trazabilidad y la ruta comodín también. El panel administrativo ofrece accesos operativos y muestra explícitamente “Gestión de usuarios: Pendiente de implementación”; por tanto, el panel no equivale a un módulo CRUD de usuarios.

1.2.4. Funcionalidades del backend: catálogo de endpoints

Se identificaron 37 mappings HTTP en los controladores revisados. Las respuestas de lectura/actualización sin otro status explícito son normalmente 200; los métodos anotados con `@ResponseStatus(CREATED)` o `ResponseEntity.created` devuelven 201. `GET /api/auth/validate` devuelve 204. Los errores de validación, estado y autorización dependen de Jakarta Validation, `ResponseStatusException` y Spring Security; no se encontró un `@ControllerAdvice` transversal.

Método	Ruta	Entrada y propósito	Respuesta / seguridad
GET	`/api/auth/validate`	Sin cuerpo; valida credenciales.	204; autenticado.
GET	`/api/auth/me`	Identidad de autenticación.	Perfil y permisos; autenticado.
GET	`/api/admin/usuarios`	Lista usuarios.	DTO sin hash; ACCESO_ADMINISTRAR.
POST	`/api/admin/usuarios`	`CreateUserRequest`; crea usuario y codifica contraseña.	201 + Location; ACCESO_ADMINISTRAR.
PATCH	`/api/admin/usuarios/{id}/estado`	Estado nuevo.	DTO usuario; ACCESO_ADMINISTRAR.
PUT	`/api/admin/usuarios/{id}/rol`	Rol existente.	DTO usuario; ACCESO_ADMINISTRAR.
GET	`/api/admin/roles`	Lista roles y permisos.	DTO roles; ACCESO_ADMINISTRAR.
POST	`/api/admin/roles`	Crea rol.	201 + Location; ACCESO_ADMINISTRAR.
PUT	`/api/admin/roles/{id}/permisos`	Reemplaza conjunto de permisos existente.	DTO rol; protege asignación de ACCESO_ADMINISTRAR; ACCESO_ADMINISTRAR.
GET	`/api/admin/permisos`	Lista permisos.	DTO permisos; ACCESO_ADMINISTRAR.

POST	`/api/admin/permisos`	Crea permiso.	201 + Location; ACCESO_ADMINISTRAR.
GET	`/api/recursos`	Lista recursos.	DTO recurso; autenticado.
POST	`/api/recursos`	`SaveResourceRequest`.	201 + Location; ACCESO_ADMINISTRAR.
PUT	`/api/recursos/{id}`	Actualiza recurso.	DTO recurso; ACCESO_ADMINISTRAR.
PATCH	`/api/recursos/{id}/estado`	Cambia estado.	DTO recurso; ACCESO_ADMINISTRAR.
GET	`/api/unidades-medida`	Lista unidades.	DTO unidad; autenticado.
GET	`/api/donaciones/catalogos/{kind}`	Tipo de catálogo, texto y página.	Página catálogo; DONACION_REGISTRAR o ACCESO_ADMINISTRAR.
POST	`/api/admin/catalogos-donaciones/{kind}`	Alta de donante/centro según tipo.	201; ACCESO_ADMINISTRAR.
POST	`/api/admin/catalogos-donaciones/unidades`	Crea unidad de medida.	201; ACCESO_ADMINISTRAR.
POST	`/api/donaciones`	`CreateDonationRequest` validado y usuario autenticado.	201 + Location con código; DONACION_REGISTRAR.
GET	`/api/donaciones/{codigoTrazabilidad}`	Consulta de código de un segmento.	DTO público sanitizado; 404 si no existe; anónimo permitido.
GET	`/api/donaciones/administracion/listado`	Filtros código/estado.	Lista DTO; ACCESO_ADMINISTRAR.

GET	`/api/donaciones/administracion/{id}`	Detalle por ID.	Detalle e historial; ACCESO_ADMINISTRAR.
POST	`/api/donaciones/administracion/{id}/recepciones`	`RegisterReceiptRequest`; cantidades e `operacionId`.	Respuesta de recepción; ACCESO_ADMINISTRAR.
GET	`/api/admin/emergencias`	Texto, estado, page y size.	Página de emergencias; ACCESO_ADMINISTRAR.
GET	`/api/admin/emergencias/{id}`	Detalle y zonas.	Detalle; ACCESO_ADMINISTRAR.
POST	`/api/admin/emergencias`	`CreateEmergencyRequest`.	201; ACCESO_ADMINISTRAR.
PUT	`/api/admin/emergencias/{id}`	Actualiza datos básicos.	Detalle; ACCESO_ADMINISTRAR.
PATCH	`/api/admin/emergencias/{id}/estado`	Cambia estado.	Detalle; ACCESO_ADMINISTRAR.
POST	`/api/admin/emergencias/{id}/zonas`	Alta de zona afectada.	201; ACCESO_ADMINISTRAR.
PUT	`/api/admin/emergencias/{id}/zonas/{zoneId}`	Actualiza zona perteneciente a emergencia.	Zona; ACCESO_ADMINISTRAR.
GET	`/api/admin/emergencias/{id}/balance`	Consulta agregada de recursos.	Lista de balance; ACCESO_ADMINISTRAR.
GET	`/api/admin/emergencias/{emergencyId}/zonas/{zoneId}/necesidades`	Lista paginada de necesidades de una zona.	Página DTO; ACCESO_ADMINISTRAR.
POST	`/api/admin/emergencias/{emergencyId}/zonas/{zoneId}/necesidades`	Recurso, cantidad, prioridad y observaciones.	201; ACCESO_ADMINISTRAR.

GET	`/api/admin/necesidades/{needId}/donaciones`	Lista líneas de donación compatibles y disponibilidad.	Página DTO; ACCESO_ADMINISTRAR.
PUT	`/api/admin/necesidades/{needId}/donaciones/{sourceId}/entrega`	Totales acumulados de asignado/entregado.	DTO con indicador repetida; ACCESO_ADMINISTRAR.
GET	`/api/admin/auditoria`	Acción, entidad, ID, usuario, fechas, page/size.	Página ordenada; ACCESO_ADMINISTRAR.

Contratos y errores: los DTO Java especifican nombres/tipos; por ejemplo, el alta de donación requiere donante, detalles no vacíos y cantidades positivas con precisión compatible con DECIMAL(14,3). La recepción exige UUID de operación y cantidades no repetidas por recurso. No se incluyen cuerpos JSON inventados. Para respuestas exitosas no explícitas se aplica el status Spring por defecto; 400/404/409 proceden de validación y excepciones de negocio, 401/403 de seguridad/autenticación.

1.2.5. Integración con MySQL

V1 crea 17 tablas relacionales y V2 agrega `recepcion_operacion`, para 18 tablas definidas en migraciones. V1 no contiene datos semilla. Variables de entorno aportan la conexión; no se incorporan credenciales al informe. Flyway gestiona el esquema y Hibernate ejecuta validación, no creación.

Grupo de tablas	Tablas verificadas	Relaciones y restricciones destacadas
Acceso	`rol`, `permiso`, `rol_permiso`, `usuario`	Rol-permiso compuesto; correo, código de rol y código de permiso únicos; usuario referencia rol.
Emergencias y catálogo	`emergencia`, `zona_afectada`, `centro_acopio`, `unidad_medida`, `recurso`	Zona pertenece a emergencia; recurso referencia unidad; fechas de emergencia verificadas.
Donaciones	`donante`, `donacion`, `detalle_donacion`, `movimiento_donacion`, `recepcion_operacion`	Código de trazabilidad único; donación exige emergencia o centro; detalle único por recurso y donación; cantidades checkeadas; operación V2 única por donación/UUID.
Necesidad y entrega	`necesidad`, `entrega`, `detalle_entrega`	Necesidad referencia zona y recurso; detalle enlaza evento, línea de donación y necesidad; cantidades no negativas. Acumulados se obtienen sumando eventos.
Auditoría	`registro_auditoria`	Usuario, entidad/ID, acción, fecha y JSON anterior/nuevo.

Recepción idempotente: `DonationReceiptService.register` bloquea la fila de donación, calcula huella SHA-256 de responsable y solicitud normalizada, busca la identidad de operación y devuelve la respuesta guardada si la operación coincide; mismo UUID con contenido o responsable distinto produce conflicto. La escritura de recepción, movimiento, auditoría y resultado de operación comparte transacción.

Asignación/entrega: el endpoint recibe totales absolutos para la pareja línea-necesidad, convierte diferencias en incrementos, rechaza disminuciones, revisa estado, recurso, emergencia, disponibilidad

recibida y tope requerido. Bajo READ_COMMITTED bloquea la donación y la necesidad antes de consultar acumulados y persiste auditoría. La repetición de los mismos totales no crea otro evento. Este diseño se observó en código; no se ejecutó una prueba de concurrencia real en MySQL.

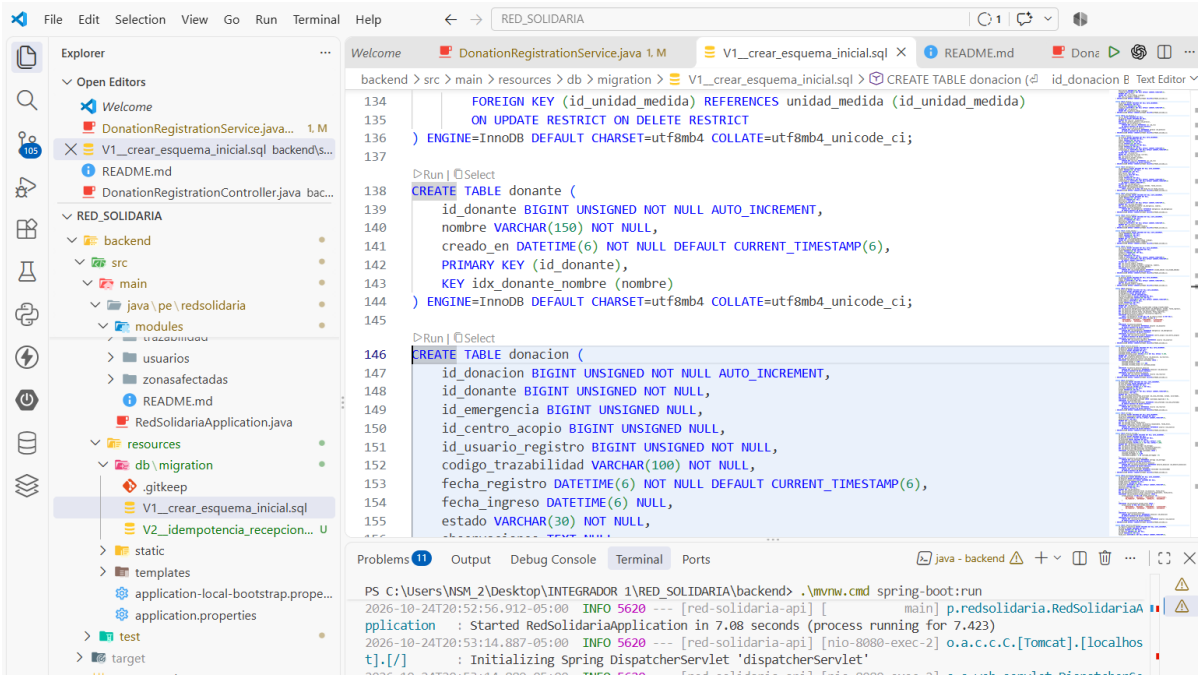


Figura 7. Evidencia E-05. Archivo `backend/src/main/resources/db/migration/V1__crear_esquema_inicial.sql`, definición de `donacion` y sus columnas/restricciones. No se ejecutó la migración durante esta tarea.

1.2.6. Trazabilidad de las funcionalidades

Funcionalidad	Componente Angular	Servicio Angular	Endpoint/controlador	Servicio backend	Persistencia
Registrar donación	`DonationRegistrationPageComponent`	`DonationAdminService.register`	POST `/api/donaciones`; `DonationRegistrationController`	`DonationRegistrationService.register`	`DonationRepository`, `DonationLineRepository`, `DonationMovementRepository`; tablas donación/detalle/movimiento.
Consultar trazabilidad	`DonationTracePageComponent`	`DonationTraceService.findByTrackingCode`	GET `/api/donaciones/{codigoTrazabilidad}`; `DonationQueryController`	`DonationQueryService.findPublicByTrackingCode`	`DonationRepository` y `DonationMovementRepository`.

Registrar recepción	DonationAdministrationDetailPageComponent.submitReceipt	DonationAdminService.registerReceipt	POST /api/donaciones/administracion/{id}/recepciones; DonationReceiptController	DonationReceiptService.register	Donación/detalles, movimiento, auditoría, operación V2.
Emergencias y balance	EmergencyManagementPageComponent, EmergencyBalanceComponent	EmergencyApiService	Endpoints bajo /api/admin/emergencias	EmergencyManagementService; balance directo vía controller/repository.	Emergencia, zona, necesidad, donaciones y entregas.
Necesidad y entrega	NeedManagementPageComponent	NeedApiService	Necesidades, fuentes y PUT de entrega bajo /api/admin	NeedManagementService, DeliveryManagementService	Need/Deliver y repositories, auditoría y tablas vinculadas.
Recursos	ResourceManagementPageComponent	ResourceApiService	/api/recursos, /api/unidades-medida	ResourceManagementService	ResourceRepository, MeasurementUnitRepository.
Auditoría	AuditPageComponent	AuditApiService.search	GET /api/admin/auditoria; AuditQueryController	AuditQueryService.search	AuditRecordRepository / registro_auditoria.
Usuarios /roles/permisos	No hay página de administración localizada.	No aplica.	/api/admin/{usuarios,roles,permisos}; AccessManagementController	AccessManagementService	Repositorios y entidades de access.

1.2.7. Reglas de negocio y validaciones

Regla	Ubicación / mecanismo	Efecto y límite de verificación
Contraseña 12–128 puntos de código y máximo 72 bytes UTF-8.	PasswordPolicy en creación REST y bootstrap.	Rechaza valores fuera de rango antes de BCrypt.
Registro requiere donante válido, destino de emergencia o centro y al menos un detalle; recurso activo, cantidades positivas/únicas y máximo 3 decimales.	CreateDonationRequest y DonationRegistrationService.validateRequest/register; V1 refuerza constraints.	Falla con 400/404/409 según caso; alta compuesta es transaccional.

Recepción debe referir recursos de la donación, ser única por recurso y no superar pendiente.	`DonationReceiptService`.	La recepción total cambia estado a RECIBIDA; parcial permanece REGISTRADA.
Reintento de recepción usa UUID y huella.	`ReceiptOperation`, `ReceiptOperationRepository`, V2 unique key.	Respuesta idéntica reutilizada; reutilización con contenido/actor distinto da 409.
Asignación no supera lo recibido real disponible ni la necesidad; donación/recurso/necesidad deben estar habilitados y compartir recurso/emergencia.	`NeedManagementService` y `DeliveryManagementService.save`; locks y READ_COMMITTED.	Evita reservar cantidad prometida en vez de recibida. Integridad concurrente no probada contra MySQL en esta tarea.
Totales de entrega son acumulados; no se reducen.	`SaveDeliveryRequest` y comparación de deltas en `DeliveryManagementService`.	Repetición idéntica es no-op; se guarda incremento y auditoría solo al cambiar.
Fechas de emergencia y longitudes de textos.	`EmergencyManagementService.validateEmergency/validateZone`; Bean Validation en DTO.	Fin no puede preceder inicio; límites de nombre, tipo y ubicación se revisan.
Donación pública no debe divulgar información personal/admin.	`PublicDonationTraceResponse` y mapping del servicio.	Omite donante, identificadores internos, observaciones y responsable.
Cambios relevantes producen auditoría.	Servicios de emergencia, necesidades, recepción y entrega; escritura dentro de transacción.	Registra actor, entidad, acción y datos anteriores/nuevos donde se implementa.
Autorización backend por permisos.	`SecurityConfiguration` y `@PreAuthorize`.	Frontend guards son complementarios; endpoints de administración exigen autoridad.

1.2.8. Estado de implementación y limitaciones

- **Implementado en código y con pruebas aisladas ejecutadas:** registro y consulta de donaciones; recepción y su servicio/dominio; emergencias; recursos; necesidades; entregas; consulta de auditoría. Los flujos se enlazan con API y repositorios en código.
- **Implementado en código, verificación parcial:** catálogos y balance de emergencia; el componente frontend se probó en la suite, pero no se ejecutó la consulta del backend contra una base real en esta ejecución.
- **Parcial en UI:** alta/listado/estado de cuentas, roles y permisos están disponibles en API, pero la pantalla de administración de usuarios se identifica como pendiente en el panel.
- **No ejecutado:** bootstrap y recuperación administrativa CLI; migraciones Flyway sobre una base; pruebas de integración MySQL/rollback/concurrencia.
- **No confirmado en el código analizado:** QR, notificaciones, aplicación móvil, reportes generales y conversión automática de unidades.
- **Esqueleto, no implementación:** carpetas españolas del backend y carpeta `common` contienen solo `.gitkeep`. El README interno de `modules` aún describe el estado inicial de marcadores y no refleja los módulos ingleses implementados.

1.2.9. Pruebas y verificación funcional

Comprobación ejecutada	Resultado	Alcance
`npm test -- --watch=false` en `frontend`	**20 archivos, 97 pruebas aprobadas.**	Suite Angular completa disponible; usa pruebas de componentes y HTTP testing, no es una prueba de despliegue ni de MySQL real.

`mvnw.cmd -Dtest=AccessManagementServiceTest,AuditQueryServiceTest,DeliveryManagementServiceTest,DonationRegistrationServiceTest,DonationReceiptServiceTest,DonationReceiptDomainTest,DonationQueryServiceTest,EmergencyManagementServiceTest,NeedManagementServiceTest,ResourceManagementServiceTest test` en backend	**10 clases, 43 pruebas aprobadas; BUILD SUCCESS.**	Servicios y dominio seleccionados. No es la suite backend completa. No incluyó tests MySQL.
Pruebas de integración condicionadas por MySQL	**No ejecutadas en esta tarea.**	No se conectó a base de datos, no se aplicó Flyway ni se modificaron filas.

La suite frontend contiene 20 archivos `.spec.ts`. El árbol backend contiene 37 archivos de prueba, entre ellos pruebas unitarias, web y algunas pruebas MySQL condicionadas. La ejecución backend realizada fue intencionalmente limitada a 10 clases sin integración para evitar tocar un servicio MySQL. Hubo advertencias de resolución de metadatos Maven no necesarios para la ejecución; la compilación y los tests seleccionados terminaron exitosamente.

1.2.10. Síntesis funcional

El código conecta páginas Angular y servicios HTTP con endpoints Spring, servicios de aplicación y tablas migradas para los flujos de donación, recepción, emergencia, necesidades, entregas, recursos y auditoría. Los casos con pruebas aisladas ejecutadas se identifican como tales; la persistencia real, el despliegue, V2 aplicada y la concurrencia permanecen sin verificar. La API de usuarios supera lo que cubre la UI actual, y varias carpetas/documentos conservan trazas de una etapa inicial.

REFERENCIAS TÉCNICAS

- README.md; backend/README.md; backend/pom.xml; frontend/package.json; frontend/angular.json.
- backend/src/main/java/pe/redsolidaria/RedSolidariaApplication.java y paquetes bootstrap, config y modules/{access,audit,deliveries,donations,emergencies,needs,resources}.
- backend/src/main/resources/application.properties y backend/src/main/resources/db/migration/V1__crear_esquema_inicial.sql, V2__idempotencia_recepciones.sql.
- frontend/src/app/app.routes.ts, app.config.ts y carpetas auth, donations, emergencies, needs, resources, audit, admin, shared.
- Documentación preexistente consultada: docs/API_ACCESO.md, docs/API_DONACIONES.md, docs/FASE_1_INTEGRACION.md. Se contrastó con el código actual.
- Capturas reales: docs/evidencias/E01_estructura_general.png a E07_rutas_angular.png (índice por nombre).